

Состязательные атаки. Задание на атаку метрики

Георгий Бычков

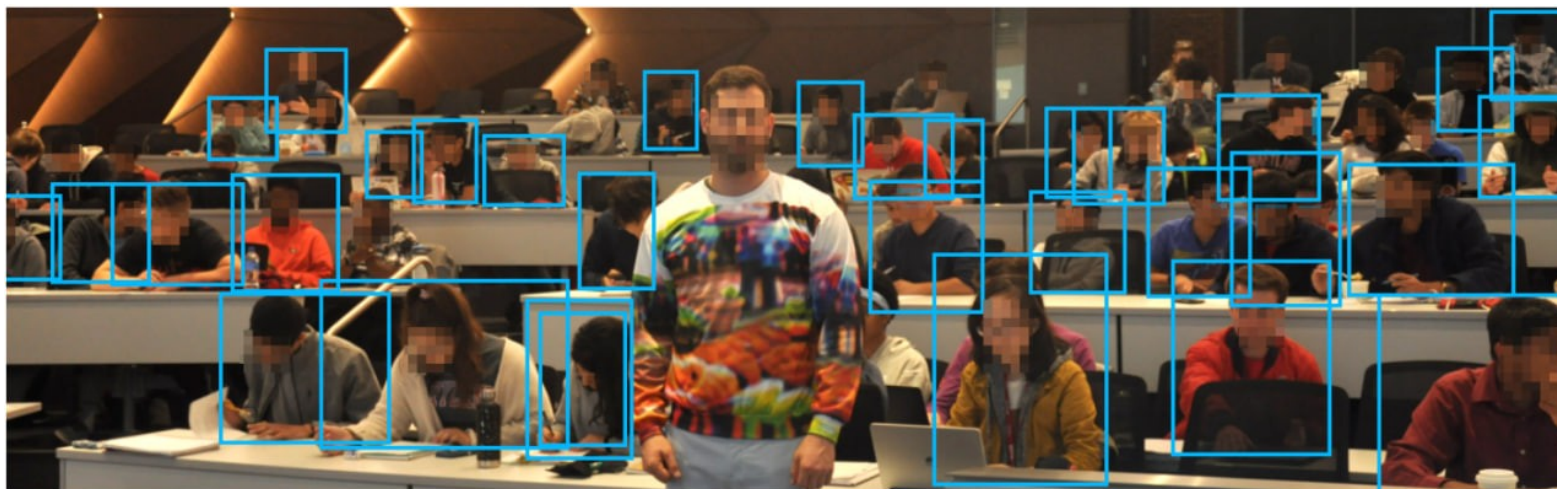
Video Group CS MSU Graphics&Media Lab

ISP RAS Research Center for Trusted Artificial Intelligence

Состязательные атаки

- Состязательные атаки могут использоваться для:
 - Тестирования устойчивости моделей (для ввода в эксплуатацию надежных нейросетей)
 - Улучшения обучения моделей (повышение обобщающей способности)
 - Понимание уязвимостей (выявление слабых мест и их исправление)
 - Повышения устойчивости (adversarial training)
 - Понимания интерпретируемости модели (на какие признаки обращает внимание)

Применение состязательных атак



Одежда, делающая человека невидимым для детекторов объектов [1]



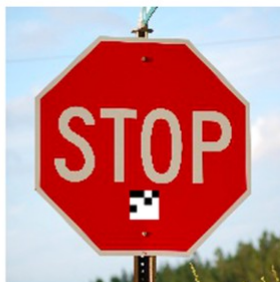
Stop

Исходное
изображение



Yield

Атакованные изображения [2]



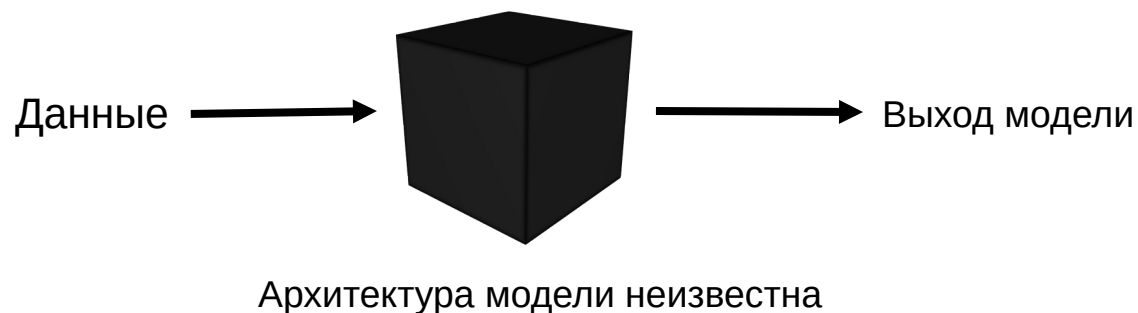
Speed Limit

1) Making an Invisibility Cloak: Real World Adversarial Attacks on Object Detectors, <https://arxiv.org/pdf/1910.14667.pdf>

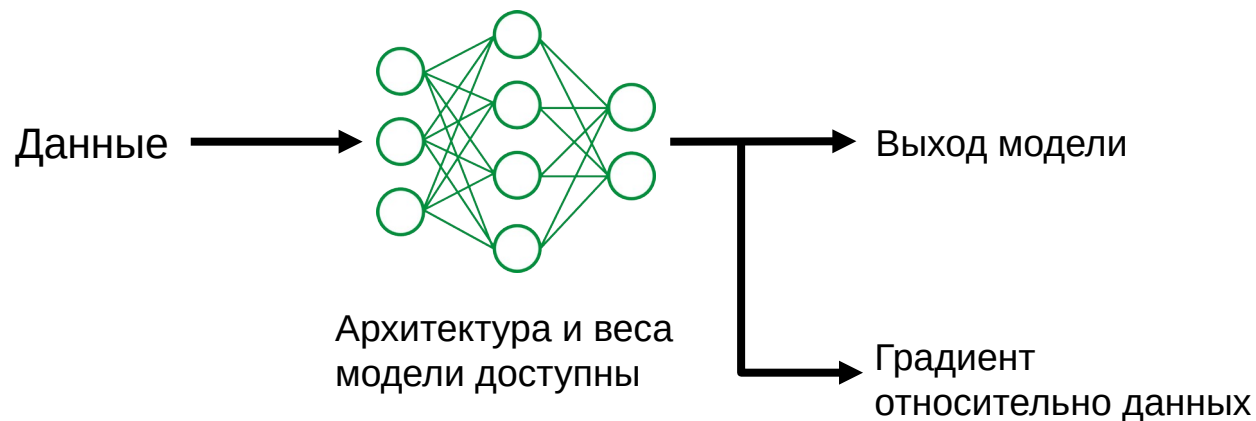
2) R. Tang et al., "An Embarrassingly Simple Approach for Trojan Attack in Deep3 Neural Networks", <https://arxiv.org/abs/2006.08826>

Виды состязательных атак

“Black Box” атака



“White Box” атака

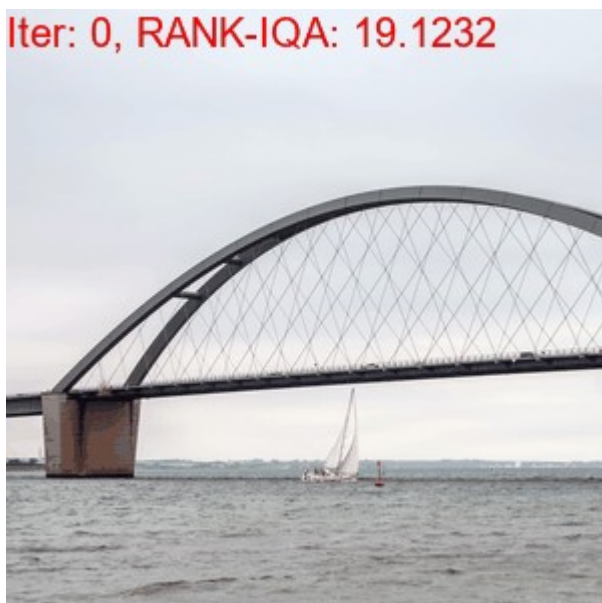


Состязательные атаки на метрики

Определение

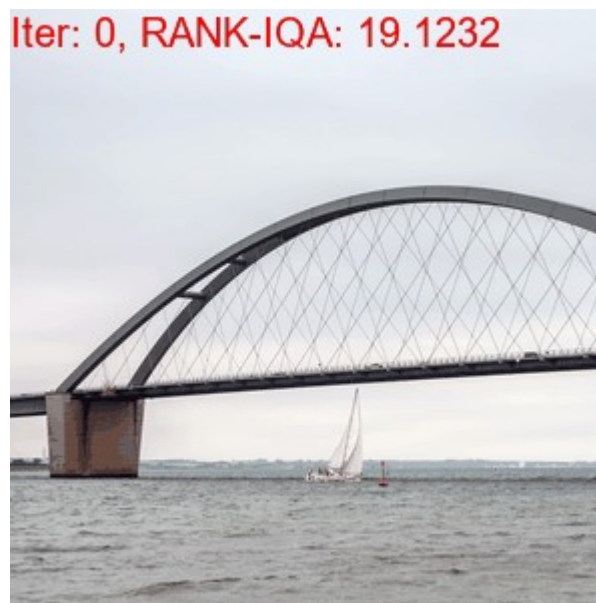


Состязательные атаки на метрики качества — манипулирование входными данными с целью увеличить или уменьшить значения метрик без соответствующего изменения реального качества изображения



Повышение метрики RANK-IQA,
атака Korhonen et al.,
изображение из датасета

NIPS2017



Понижение метрики RANK-IQA,
атака Korhonen et al.,
изображение из датасета
NIPS2017

Состязательные атаки на метрики

Непреднамеренный взлом



Использование нестабильных метрик в качестве компонент функции потерь может привести к "непреднамеренному взлому"

Возможные решения:

- использование простых стабильных метрик
- использование ансамблей метрик
- **использование стабильных метрик, основанных на машинном обучении**



DISTS отдаёт предпочтение кадрам с повышенной яркостью. Использование DISTS в функции потерь при обучении нейросетей может привести к генерации изображений с повышенной яркостью.

Состязательные атаки на метрики

Непреднамеренный взлом



Использование нестабильных метрик в качестве компонент функции потерь может привести к "непреднамеренному взлому"

Возможные решения:

- использование простых стабильных метрик
- использование ансамблей метрик
- **использование стабильных метрик, основанных на машинном обучении**



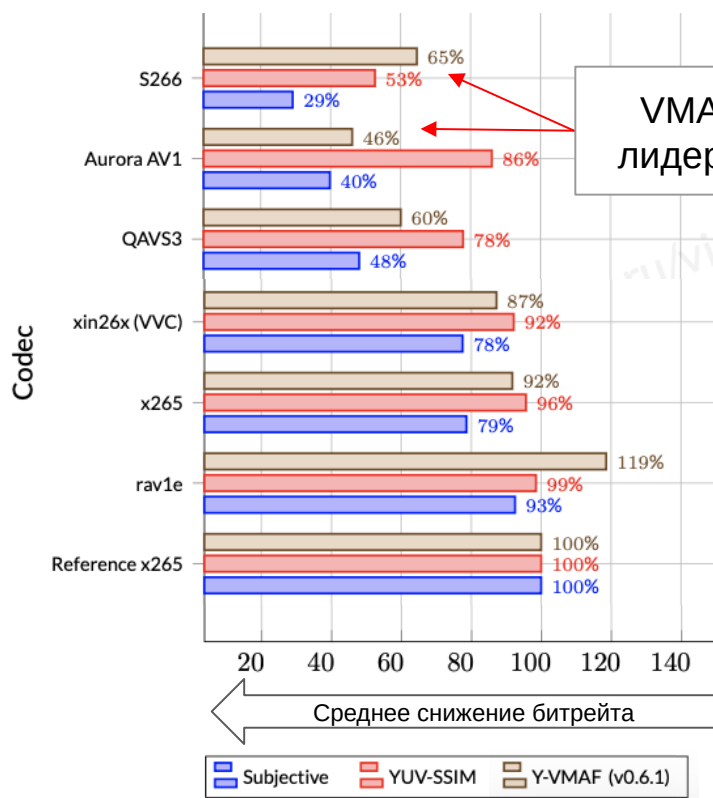
Результат работы нейросетевого метода повышения разрешения BasicVSR++, обученного с ансамблем метрик в функции потерь, включающим в себя LPIPS, NIMA, CLIP-IQA

Состязательные атаки на метрики

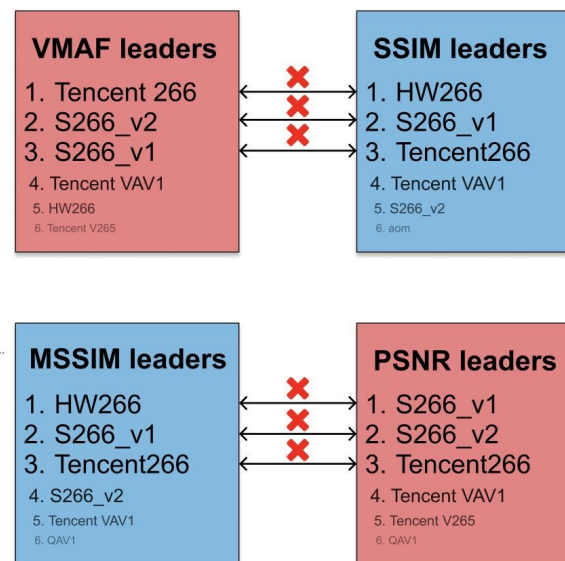
Реальный пример в сравнении



В сравнениях/соревнованиях результат сильно зависит от выбранной метрики



VMAF второго энкодера лучше лидера субъективного сравнения



Состязательные атаки на метрики

Пример атаки на VMAF



1. Препроцессинг видео
2. Сжатие
3. Вычисление VMAF
4. Использование генетического алгоритма для оптимизации параметров

Видео не визуально не поменялось, но
VMAF начал оценивать качество выше



- Пусть дана дифференцируемая скалярная функция $f: \mathbb{R}^n \rightarrow \mathbb{R}$
- Ее градиентом в точке $p = (x_1, \dots, x_n)$ называется вектор $\nabla f: \mathbb{R}^n \rightarrow \mathbb{R}^n$

$$\nabla f(p) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(p) \\ \vdots \\ \frac{\partial f}{\partial x_n}(p) \end{bmatrix}$$

- Градиент указывает на направление наибольшего роста функции

FGSM

Пример для классификаторов

l_∞ ограничение

$+ .007 \times$

x

“panda”
57.7% confidence

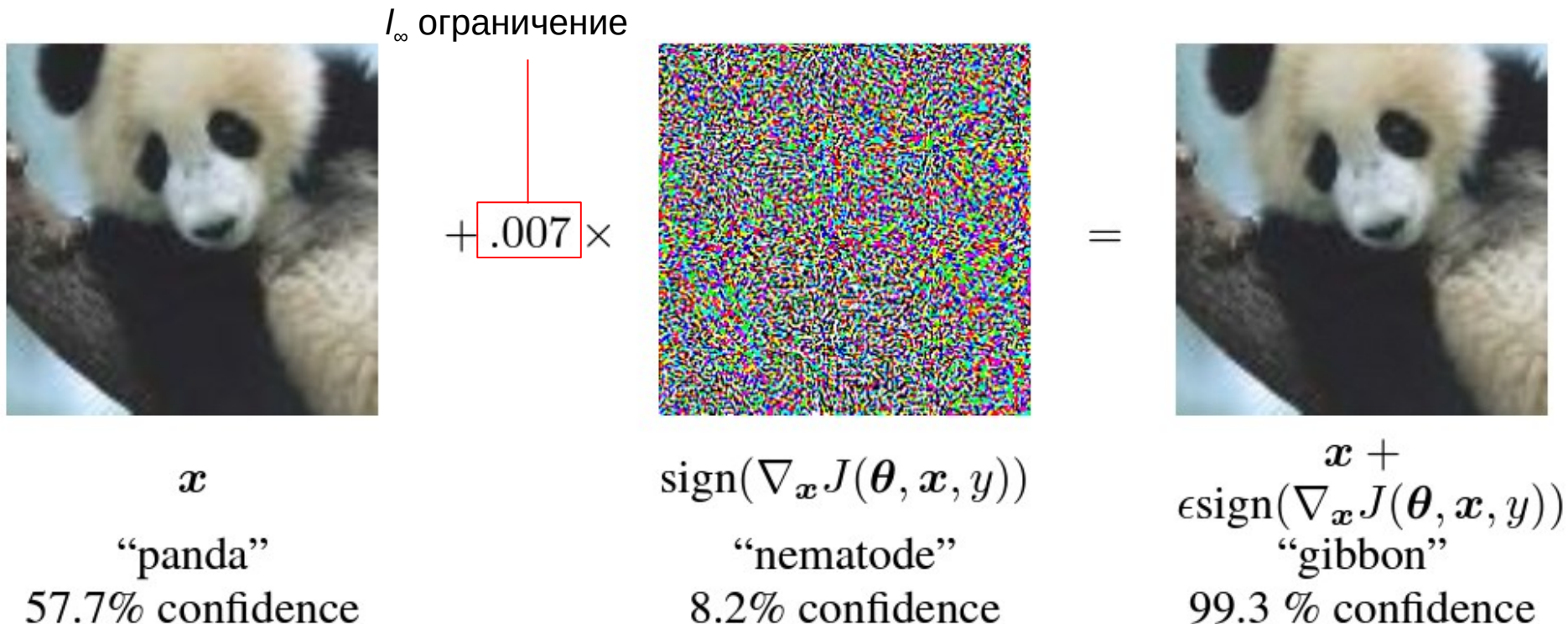
$\text{sign}(\nabla_x J(\theta, x, y))$

“nematode”
8.2% confidence

$=$

$x + \epsilon \text{sign}(\nabla_x J(\theta, x, y))$

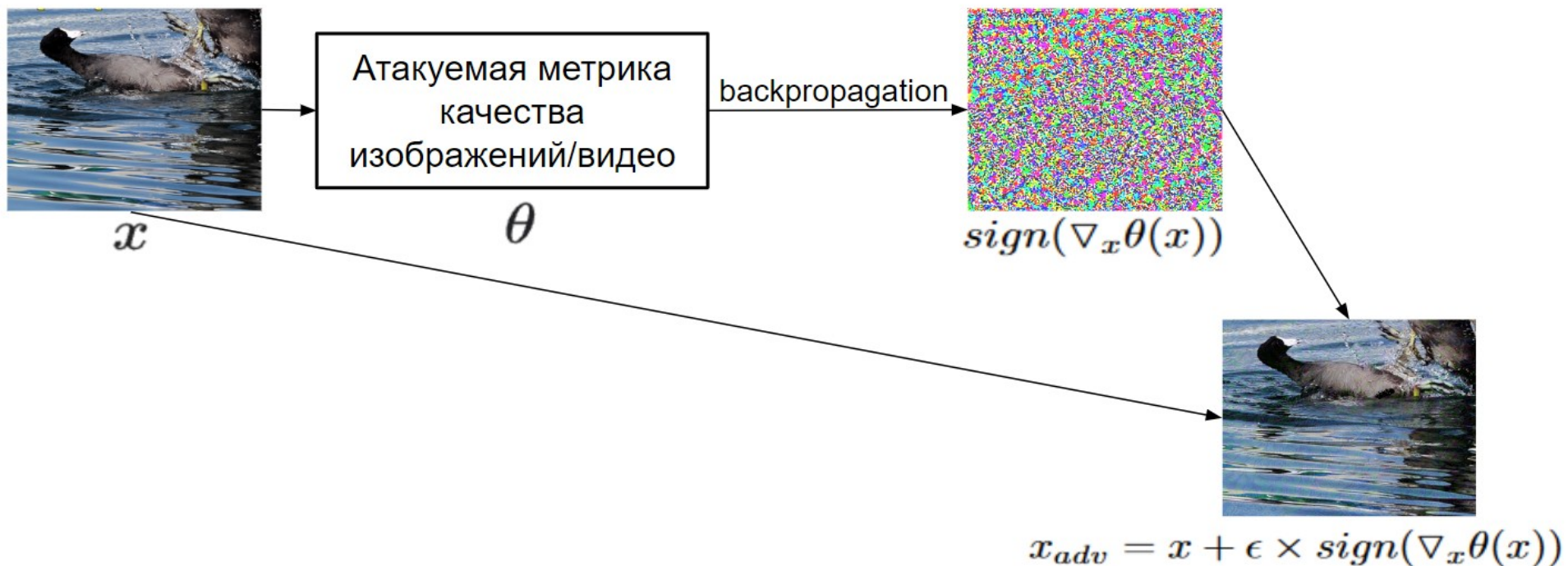
“gibbon”
99.3 % confidence



I. Goodfellow, J. Shlens and C. Szegedy,
“Explaining and Harnessing Adversarial Examples,”
in *ICLR*, 2015

FGSM

Общий вид



- Обычно для генерации состязательных примеров используется $J(\theta, \mathbf{x}, y)$, которая используется при обучении нейросети
- В случае безреференсных метрик обычно используется $L(f_\omega(x')) = \frac{f_\omega(x')}{\text{diam}(f_\omega)}$ $\text{diam}(f_\omega) = \sup_{x, z \in X} \{|f_\omega(x) - f_\omega(z)|\}$
 $f_\omega(x)$ — значение метрики на изображении X

I-FGSM

В статье предложен итеративный подход, улучшающий FGSM: $x_0^* = x$, $x_{t+1}^* = x_t^* + \alpha \cdot \text{sign}(\nabla_x J(x_t^*, y))$
где $\alpha = \epsilon/T$, либо произвольная с клипингом x_t



clean

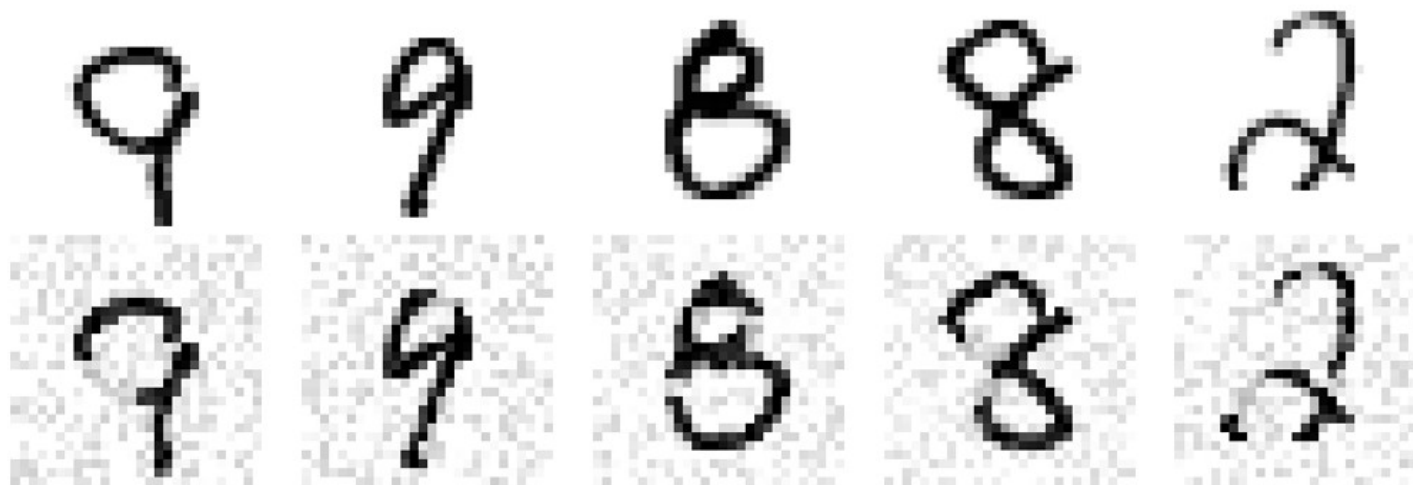


attacked

PGD

- Предлагается модификация I-FGSM:

$$x^{t+1} = \Pi_{x+\mathcal{S}} (x^t + \alpha \operatorname{sgn}(\nabla_x L(\theta, x, y)))$$
- Отличия от I-FGSM:
 - Случайная инициализация начального решения
 - Проекция на шар, вместо клипинга



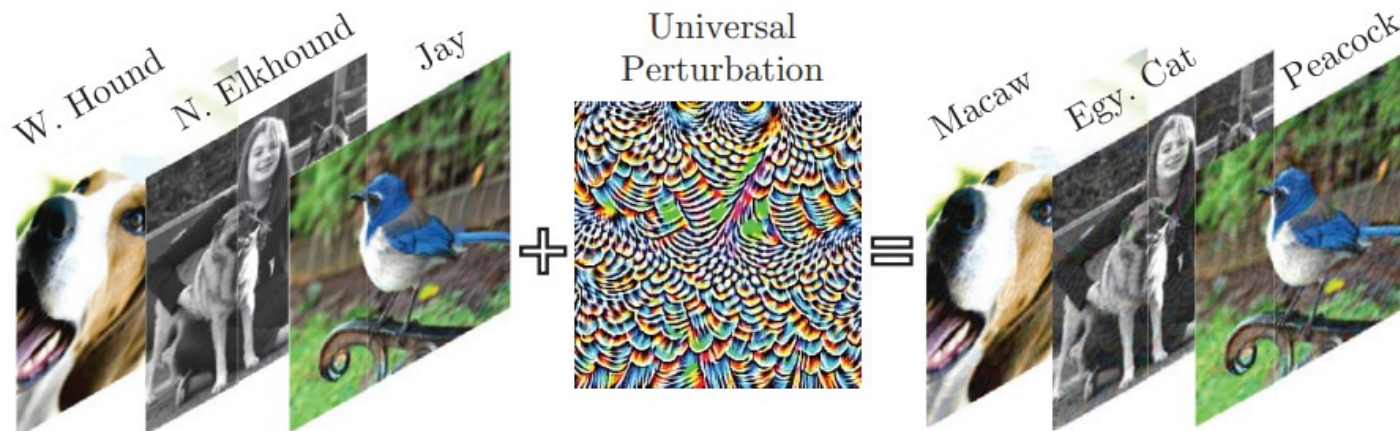
Natural: 9	Natural: 9	Natural: 8	Natural: 8	Natural: 2
Adversarial: 7	Adversarial: 4	Adversarial: 5	Adversarial: 3	Adversarial: 3

UAP атака

Введение

Algorithm 2 Stochastic gradient for universal perturbation

```
for epoch = 1 ...  $N_{ep}$  do  
  for minibatch  $B \subset X$  do  
    Update  $\delta$  with gradient variant  $\delta \leftarrow \delta + g$   
    Project  $\delta$  to  $\ell_p$  ball  
  end for  
end for
```



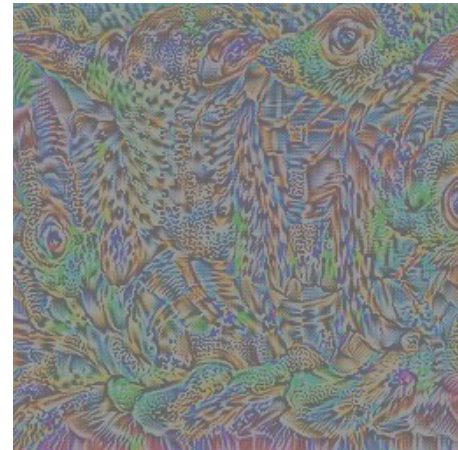
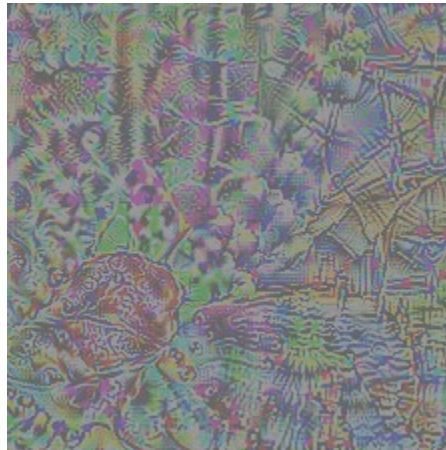
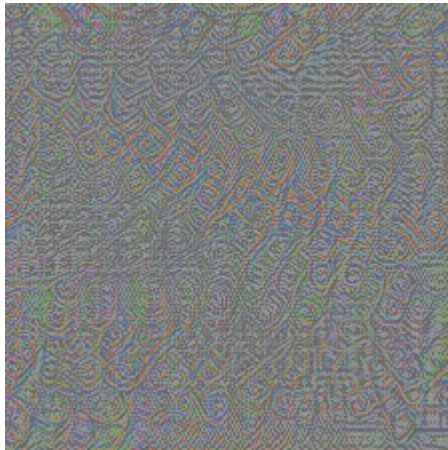
UAP атака

Реализация в случае метрик

- Используя измененную функцию потерь можно сделать UAP атаку

$$L(f_{\omega}(x')) = \frac{f_{\omega}(x')}{\text{diam}(f_{\omega})}$$

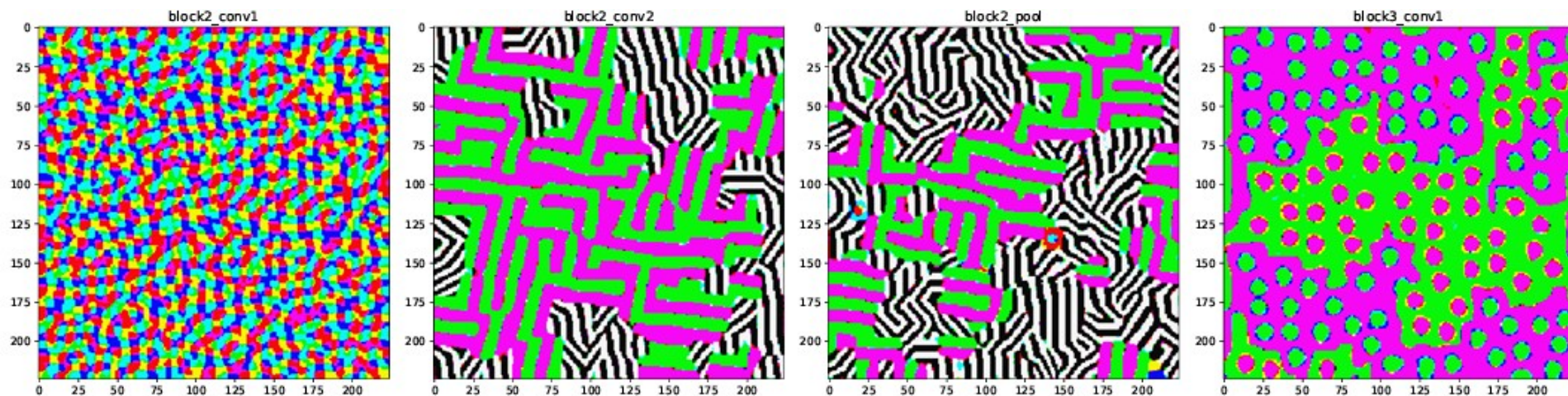
- В данных примерах UAP обучался с ограничением на l_{∞} норму



UAP атака

Математический подход

- Заметим, что $f_i(x + \varepsilon) - f_i(x) \approx J_i(x)\varepsilon$, где $f_i(x)$ — выход i слоя, а якобиан i слоя нейросети — $J_i(x) = \left. \frac{\partial f_i}{\partial x} \right|_x$
- Обучение UAP:
$$\sum_{x_j \in X_b} \|J_i(x_j)\varepsilon\|_q^q \rightarrow \max, \quad \|\varepsilon\|_p = L$$
- Задача решается с помощью степенного метода и является (p,q)-сингулярным вектором матрицы



V. Khrulkov and I. Oseledets, "Art of singular vectors and universal adversarial perturbations," in CVPR, 2018

Дифференцируемый JPEG

Напоминание JPEG



- JPEG сам по себе не дифференцируемый из-за операции квантования
- Хочется иметь возможность строить пайплайн с JPEG, чтобы можно было делать backpropagation

Дифференцируемый JPEG

Решение



- Использовать дифференцируемые аппроксимации. Например:
 - <https://github.com/mlomnitz/DiffJPEG>
 - <https://github.com/necla-ml/Diff-JPEG>
- Решения достаточно близки к стандартному JPEG и разница почти незаметна
- Пример использования для Shin et al:

```
from DiffJPEG import DiffJPEG
jpeg = DiffJPEG(height=224, width=224, differentiable=True, quality=80)
compressed_diff_jpeg = jpeg(im_tensor)
```

✓ 0.0s

Python

- Пример использования для Reich et al:

```
1 from diff_jpeg import DiffJPEGCoding
2
3 diff_jpeg_module = DiffJPEGCoding(ste=True)
4 image_coded_diff = diff_jpeg_module(img_tensor*255, jpeg_quality=torch.tensor([80], device=img_tensor.device))
```

✓ 0.0s

Python

Задание по атакам на метрики

Описание



- Дан обучающий датасет из изображений
- Метрика качества изображений — [PaQ-2-PiQ](#)
- Необходимо сделать 2 атаки: атакующую одно изображение (например, FGSM) и UAP
- Атакованные изображения должны быть устойчивы к сжатию

Задание по атакам на метрики

Оценивание (1)

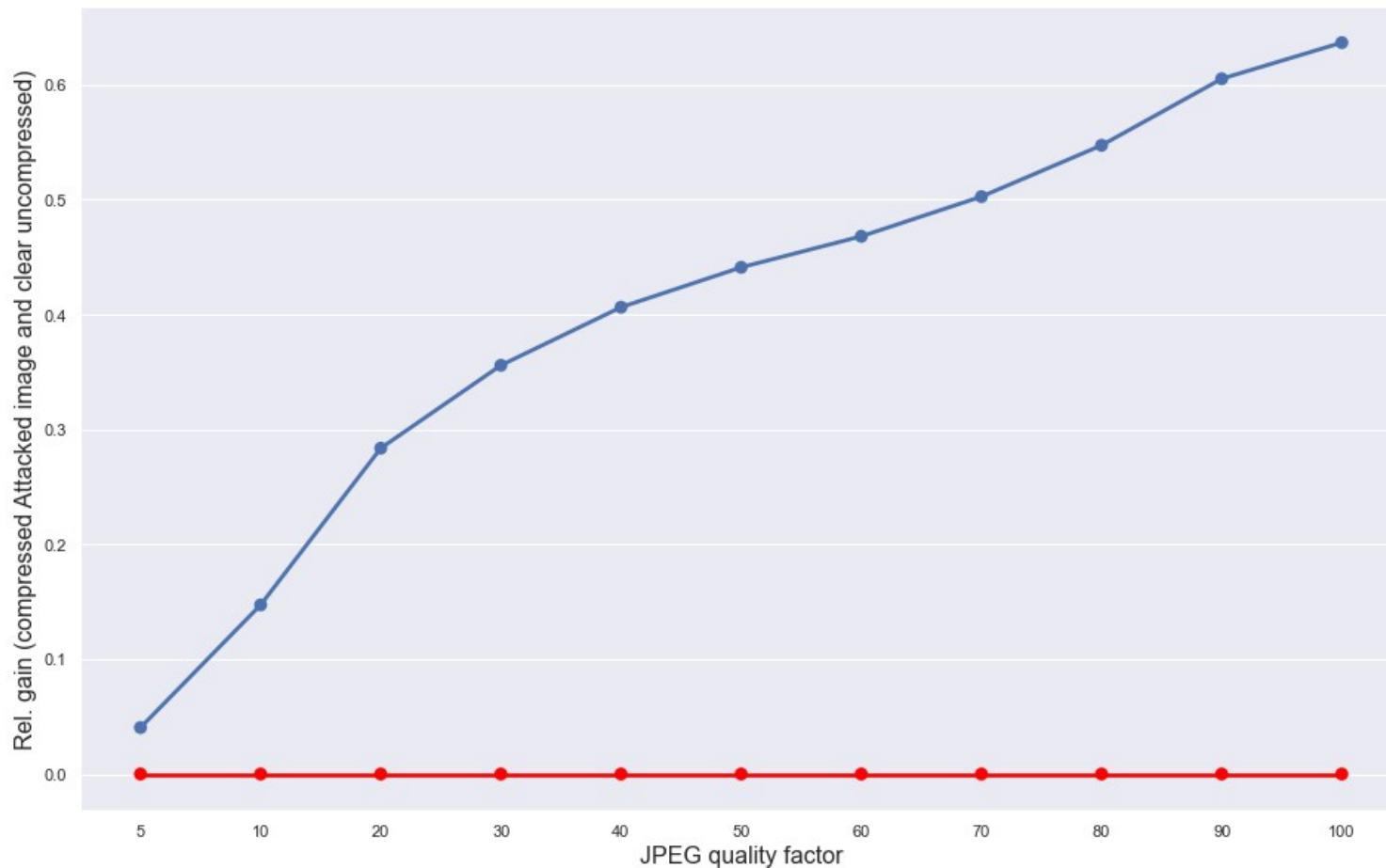


Для каждого изображения вычисляется площадь под графиком с абсциссой JPEG quality factor и ординатой Rel. gain (отношение значения PaQ-2-PiQ на проатакованном изображении к чистому, минус 1 для центрирования) для каждого

$$\varepsilon = \left\{ \frac{2}{255}, \frac{4}{255}, \frac{8}{255}, \frac{10}{255} \right\}$$

Задание по атакам на метрики

Пример графика для одного ε



Задание по атакам на метрики

Визуализация сжатия



Сжатие атакованного изображения



Сжатие исходного изображения

Задание по атакам на метрики

Оценивание (2)



- Для каждого изображения берется среднее арифметическое этих площадей
- Итоговый результат получается как среднее арифметическое полученных для каждого изображения величин

Задание по атакам на метрики

Идеи по улучшению результатов



- Общее
 - Поиграться с оптимизаторами и их параметрами (вычислительно трудно)
 - Поиграться с лосс-функцией (придумать составную, можно постараться сделать атаку менее заметной)
 - Добавить в обучение дифференцируемый JPEG
- FGSM
 - Попробовать I-FGSM, PGD, другие методы
- UAP
 - Попробовать поискать другие подходы (например, [тут](#))

Задание по атакам на метрики

Структура задания



- `test.py` — скрипт для тестирования атаки
- `iterative.py` — файл решения
- `uap.py` — файл решения
- `uap_train_script.py` — вспомогательный скрипт для запуска обучения UAP добавки
- `paq2piq_standalone.py` — код для PaQ-2-PiQ. В файле объявлен класс `MetricModel`, реализующий метрику как `torch.nn` модуль с методом `forward`, поддерживающий `backpropagation`.
- `weights/` — веса для метрики PaQ-2-PiQ
 - `RoIPoolModel.pth`
- `training_dataset/` — обучающий датасет для UAP атаки (<https://dione.gml-team.ru:5001/sharing/63K2wplLJ>)
- `public_dataset/` — публичная часть датасета для тестов (<https://dione.gml-team.ru:5001/sharing/fnB9HbuB9>)

```
python uap_train_script.py --path_train ./training_dataset --save_path ./uap_paq2piq.png  
--model_weights ./weights/RoIPoolModel.pth --batch_size 8 --device cuda:0
```

```
python test.py --attack_type uap --uap_train_path ./uap_paq2piq.png --csv_results_dir  
results_dir --device cuda:0 --dataset_path ./public_dataset  
--model_weights ./weights/RoIPoolModel.pth
```